

Verifier-Guided Evaluation of LLM-Generated Network Configurations: a Paired Study with Shared Initial Candidates

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We preregistered and executed a paired confirmatory study (CAND-2026-001) that measured whether a verifier-triggered guarded pipeline (generate → validate → repair, ≤1 repair) reduces deterministic-invariant violations relative to single-shot initial generation for intent→configuration NetOps tasks. Using a deterministic synthetic task generator and a deterministic network verifier, we ran 200 preregistered tasks under shared_initial_candidate semantics with the hosted model gpt-5-mini (execution/model_configuration.json records temperature = null/unset). Baseline configurations passed the verifier in 199/200 tasks (99.5%); the guarded pipeline passed 200/200 (100%). Paired absolute risk difference (guarded - baseline) = 0.005; contingency counts n11=199, n10=1, n01=0, n00=0; exact two-sided McNemar $p = 1.0$; paired-bootstrap 95% percentile CI = [0.00, 0.015] (10,000 resamples, seed 1729). The single guarded improvement arose from one verifier-triggered repair that corrected a multi-step ordering violation. We report preregistration, full execution accounting (201 model calls: 200 shared initial + 1 repair), paired analysis, representative diagnostic artifacts, and bounded limitations grounded in the archived execution bundle.

I. INTRODUCTION

Generative large language models (LLMs) are increasingly proposed to translate high-level operator intent into device configurations for network operations (NetOps). Operational correctness for such workflows requires generated configurations to satisfy deterministic invariants (reachability, policy compliance, workflow ordering, and group-level safety). Prior prototype systems and benchmarks illustrate feasibility but leave open questions about how much deterministic verification and bounded repair alter acceptance rates when the identical initial sample is reused across comparison conditions.

This paper reports a preregistered paired experiment (CAND-2026-001) designed to isolate the marginal effect of a verifier-triggered automated repair on an identical generated candidate. Under shared_initial_candidate semantics the same single initial generation is deterministically reused by both baseline and guarded episodes; any paired difference therefore arises solely from the permitted validator-triggered repair. The primary estimand is the paired difference in per-task binary acceptance by a deterministic verifier (paired_success_rate_difference_guarded_minus_baseline). Because shared_initial_candidate semantics were enforced, this estimand measures a repair-only effect and does not capture potential benefits from independent guarded first-pass generations, constrained prompts, or multi-sample selection.

Contributions. (1) A preregistered, fully archived paired experiment executing 200 intent→configuration tasks with a deterministic verifier and a hosted LLM (gpt-5-mini) under shared_initial_candidate semantics. (2) Complete execution accounting and deterministic reconciliation showing 200 shared initial generations and 1 repair call (201 model calls total). (3) Artifact-grounded confirmatory analysis reporting baseline pass 199/200 and guarded pass 200/200 with contingency counts (n11=199, n10=1, n01=0, n00=0), paired risk difference 0.005, exact two-sided McNemar $p = 1.0$, and bootstrap 95% CI = [0.00, 0.015] (10,000 resamples, seed 1729). (4) A localized failure diagnosis for the single repaired pair using archived validator traces. (5) Detailed reproducibility guidance and bounded limitations tied to archived artifacts and reconciliation records.

II. RELATED WORK

Recent work on LLM-driven NetOps shows convergent interest in intent→configuration synthesis, verification-aware pipelines, and tool-augmented agent designs, but the supplied verified records reveal complementary strengths and limitations that motivated our paired design. NetConfEval and related intent→configuration prototypes demonstrate that LLMs can produce workable device configurations from natural-language intent and that task-style benchmarks are useful for early evaluation; these studies report feasibility and dataset-limited performance measures but do not uniformly integrate a deterministic verifier as the final oracle for acceptance decisions [1]. Deterministic configuration verification methods (e.g., formal and symbolic reachability/policy-check approaches) provide a clear operational oracle for syntactic and semantic invariants and have been developed in the NetOps literature; these verification techniques form the natural oracle used in our experiment and highlight why a verifier-centric evaluation is defensible for configuration-synthesis tasks [2].

Across adjacent domains, generate→validate→repair patterns have empirical support: test-in-the-loop program-repair and synthesis-with-tests pipelines show that validating generated artifacts against executable checks and iteratively repairing failures materially improves correctness in software-engineering tasks [4]. Those demonstrations motivate guarded NetOps pipelines but are not direct evidence of net effects in network-configuration contexts because NetOps invariants and multi-device sequencing introduce distinct dependency

patterns. Tool-augmented agent and model-context approaches further extend capability by granting controlled access to specialized solvers, validators, or execution environments; evaluations of such tool-augmented designs argue they enable more complex, verifiable behavior but also point out engineering and interoperability challenges when integrating heterogeneous oracles [6].

Survey and reporting-guideline literature places additional context around rigor and reproducibility: broad LLM surveys synthesize model capabilities and limitations at scale and underscore the importance of careful experimental reporting and preregistration when evaluating generative systems [5]. Reporting guidelines for studies that use LLMs emphasize explicit documentation of sampling, validation, and analysis choices to avoid overclaiming—this influenced our preregistration and artifact-archiving decisions [3]. Finally, focused work on hallucination detection and validator gating in generative systems highlights common failure modes (confabulation, ordering errors, and adversarially induced fabrications) and motivates verifier gates as necessary but not sufficient mitigations; such analyses support our guarded pipeline as a targeted, bounded intervention rather than a general cure [7].

Synthesis. The supplied and verified literature collectively supports three points: LLM-based intent→config pipelines are feasible and actively prototyped; generate→validate→repair is a promising architectural pattern whose empirical demonstrations to date are stronger in code and testable artifacts than in NetOps-specific verifier studies; and rigorous reporting and verifier-grounded evaluations are necessary to interpret operational claims. Our paper addresses a narrower, previously underexplored empirical gap in the supplied records: a reproducible paired measurement of the marginal (repair-only) effect when the identical initial candidate is reused across baseline and guarded conditions. By enforcing shared_initial_candidate semantics and archiving the verifier traces, the present study isolates that repair-only estimand and complements prior prototype and generate–validate–repair demonstrations [1],[2],[4],[6],[3],[5].

III. METHODOLOGY

Preregistration and design freeze. We preregistered study CAND-2026-001 before execution. The preregistration fixed the primary estimand, the deterministic sampling of 200 tasks drawn by deterministic_netops_task_generator_v5 (rule-based index selection documented in the manifest), generation_semantics = shared_initial_candidate (one sampled initial generation per task deterministically reused), and allowed at most one automated repair call per task. The preregistration specified the confirmatory test (exact two-sided McNemar via exact binomial tail), the primary failed-call handling (complete_pair_primary), a sensitivity analysis treating persistent unparseables as failures, and a power rationale aiming to detect ~10–15 percentage-point paired increases at N=200 under mid-range baseline assumptions (~45–65%). The preregistration artifact is archived (preregistration_sha256 in execution manifest).

Task bank, difficulty encoding, and representative statistics. Tasks were generated deterministically (deterministic_netops_task_generator_v5) and targeted high-difficulty workflow patterns (very_hard and extreme). Each task manifest encodes initial_state, expected_final_state, required_sequence (ordered command list), allowed_touched_fields, workflow_policies (e.g., at-least-one-up constraints), difficulty metadata (changed_interface_count, dependency_rule_count, required_command_count), and a normalized reference_answer. In the archived representative sample required_command_count spans 3–9 and changed_interface_count up to 3; the single baseline failure occurred on an extreme composite task with required_command_count = 9, consistent with concentrated failure risk on higher-complexity tasks (see representative_task entries).

Model configuration and sampling semantics. The declared requested model was gpt-5-mini (execution/model_configuration.json). That file records temperature = null (unset). We explicitly note that temperature = null/unset is not evidence of deterministic sampling or temperature = 0; nondeterministic hosted-model sampling remains possible. Provider accounting (deterministic_reconciliation.json) reports stage_counts = {shared_initial_generation: 200, repair: 1} and hosted_model_call_event_count = 201; cache_miss_count = 201, cache_hit_count = 0 indicates fresh generation events for the shared-initial calls.

Deterministic verifier and scoring. The deterministic verifier deterministic_netops_validator_v5 served as the oracle. It parses candidate configurations, enforces per-device allowed-field constraints, enforces required_sequence ordering, evaluates group invariants (e.g., final_group_constraints), and returns a binary full_intent_constraint_validation score of 1 iff all encoded invariants pass. Per-episode validator_trace artifacts include validation_before and validation_after states, parsed_command_count, required_command_count, observed_touched_field_count, violation_codes, and a boolean valid flag. These verifier traces are archived under execution/scoring/ and execution/responses/ and constitute the authoritative evidence for per-episode scoring.

Paired execution semantics and failure handling. Under shared_initial_candidate semantics baseline and guarded episodes for each pair begin from the identical sampled candidate; any paired difference therefore arises solely from the permitted verifier-triggered repair. The missingness_plan specified a deterministic sanitizer for minor formatting issues and a sensitivity analysis treating persistent unparseables as failures. In the archived run there were no persistent unparseables and no preregistered exclusions; terminal_error_type fields are null for all episodes in scoring artifacts.

Statistical analysis and exact McNemar implementation. The primary confirmatory test is the exact two-sided McNemar computed from discordant pair counts n10 (guarded=1, baseline=0) and n01 (guarded=0, baseline=1). We implement the exact two-sided McNemar via the exact binomial tail: with n

= $n_{10} + n_{01}$ and $k = n_{10}$, the two-sided p-value is computed as $p = 2 * \min\{ \text{BinomCDF}(k; n, 0.5), 1 - \text{BinomCDF}(k - 1; n, 0.5) \}$ with conventional handling for $k = 0$. For our observed discordants ($n = 1, k = 1$): $\text{BinomCDF}(1; 1, 0.5) = 1$ and $1 - \text{BinomCDF}(0; 1, 0.5) = 0.5$, so $p = 2 * \min(1, 0.5) = 1.0$. Paired-bootstrap percentile confidence intervals for the absolute risk difference were computed with 10,000 resamples and `bootstrap_seed = 1729`; implementation parameters and seed are archived in `analysis/analysis_log.jsonl`.

IV. RESULTS

Execution accounting and provider audit. The run started at 2026-09-01T16:03:05.491039+00:00 and completed at 2026-09-01T16:32:13.298536+00:00 (≈ 29.2 minutes wall-clock). The execution manifest records `planned_episode_count = 400` and `completed_episode_count = 400` with `failed_episode_count = 0`. Provider-call accounting (`deterministic_reconciliation.json`) reports `hosted_model_call_event_count = 201`, with `stage_counts = {shared_initial_generation: 200, repair: 1}` and `condition_counts` reported as `{shared: 200, guarded: 1}` under the adapter’s stage/condition accounting semantics. Cache statistics show `cache_miss_count = 201` and `cache_hit_count = 0`, consistent with fresh shared-initial generations for all tasks plus a single repair call. Dividing the wall-clock duration by the 201 hosted-model calls yields an approximate average model-call latency of 8.7 seconds per call (simple elapsed-time / call count calculation from archived timestamps and `model_calls_used`).

Primary confirmatory outcomes (archived CSV artifacts). The archived condition summary (`analysis/condition_summary.csv`) records baseline successes = 199, baseline failures = 1 (`success_rate = 0.995`) and guarded successes = 200, guarded failures = 0 (`success_rate = 1.000`). The paired contingency table (`analysis/paired_contingency_table.csv`) contains the exact cell counts used by the confirmatory test: $n_{11} = 199$ (both pass), $n_{10} = 1$ (guarded pass only), $n_{01} = 0$ (baseline pass only), $n_{00} = 0$ (both fail). From these counts the observed paired absolute risk difference (guarded - baseline) = 0.005.

Exact-test and interval quantification. Applying the exact two-sided McNemar (binomial-tail formulation) to the discordant counts ($n = n_{10} + n_{01} = 1, k = n_{10} = 1$) yields $p = 1.0$ as computed in Methods. To quantify plausible effect sizes we computed a paired-bootstrap percentile 95% confidence interval for the absolute risk difference using 10,000 resamples with `bootstrap_seed = 1729` (`analysis/analysis_log.jsonl`). The resulting 95% percentile CI is [0.00, 0.015], giving a bounded estimate consistent with at most small absolute improvements under the repair-only estimand given the observed data.

Missingness, contamination, and deterministic reconciliation. The run recorded no preregistered exclusions and no persistent unparsables; `analysis/missingness_summary.csv` shows `complete_pairs = 200` and zeros for all other missingness categories. `analysis/contamination_summary.csv` contains no flagged

contamination entries. The deterministic reconciliation artifact (`analysis/deterministic_reconciliation.json`) reports `pair_accounting_consistent = true` and `all_deterministic_consistency_checks_passed = true`, confirming internal accounting consistency between provider-call logs, `stage_counts`, and analysis inputs.

Localized failure diagnosis and repair evidence. The guarded pipeline invoked exactly one automated repair (`stage_counts` shows `repair = 1`). Validator traces for the repaired pair (archived under `execution/scoring/`) indicate an extreme composite task whose manifest metadata records `required_command_count = 9` and `changed_interface_count = 3`. The shared initial candidate for that pair violated the manifest’s `required_sequence` ordering constraint; `validation_before.violation_codes` and `validation_before.normalized_configuration` recorded the pre-repair mismatch. The repair interaction produced a localized reordering of commands that matched the manifest’s `required_sequence` and yielded `validation_after.valid = true`. The evidence supporting this sequence of events is entirely artifact-grounded: the per-episode `validator_trace` fields (`validation_before.violation_codes`, `parsed_command_count`, `required_command_count`, `normalized_configuration`, `repair_applied` flag, and `validation_after.*`) and the repair provider trace are archived and indexable. These traces show that the repair fixed an ordering/dependency violation expressible in the task manifest’s invariants rather than a syntactic parsing error.

Exploratory diagnostics by difficulty metadata. Representative task manifests available in the execution bundle document `required_command_count` values ranging from 3 up to 9 and difficulty levels labeled `very_hard` or `extreme`. The single baseline failure (the repaired pair) is associated with the highest recorded complexity in the representative sample (`required_command_count = 9`, `changed_interface_count = 3`), consistent with the preregistered exploratory hypothesis that multi-device sequencing and dependency complexity concentrate residual failures. No other tasks in the archived representative subset exhibited violations requiring repair; many `very_hard` tasks (`required_command_count` in the mid-range) were validated successfully without repair.

Practical reproducibility pointers for confirming results. All artifacts needed to reproduce the primary numbers are archived and referenced by the execution manifest: `analysis/paired_contingency_table.csv` and `analysis/condition_summary.csv` contain the per-condition counts; `analysis/analysis_log.jsonl` contains the bootstrap parameters (`resamples=10000`, `seed=1729`); `deterministic_reconciliation.json` and `execution/model_configuration.json` record provider-call accounting and declared model fields (including `temperature = null`); `execution/responses/` and `execution/scoring/` contain the shared-initial responses and per-episode `validator_trace` objects demonstrating `validation_before` and `validation_after` states and the single repair interaction. Re-running the archived `paired_binary_analysis_v1` executor on the archived

input_results_sha256 reproduces the confirmatory outputs (analysis/results.json) without recomputation of model calls.

Summary of empirical evidence. The archived run shows that under shared_initial_candidate semantics (the same sampled candidate reused by both conditions) and with the chosen synthetic task bank and deterministic verifier, nearly all initial single-shot candidates already satisfied the encoded deterministic invariants (199/200). Allowing a single verifier-triggered bounded repair corrected one manifest-expressed ordering violation in an extreme composite task, producing the observed paired improvement ($n_{10} = 1$). The small number of discordants places the inferential regime in a sparse-discordant setting where the exact McNemar test yields $p = 1.0$ but the bootstrap CI bounds the plausible absolute improvement to at most ~ 1.5 percentage points under the paired repair-only estimand.

V. DISCUSSION

Interpretation under shared_initial_candidate semantics. Because baseline and guarded conditions reuse the identical sampled candidate per pair, observed differences isolate the marginal effect of permitting an automated verifier-triggered repair. The experiment therefore quantifies a repair-only effect and does not evaluate other guarded variants that would permit independent guarded first-pass generations or constrained prompting. We reiterate this estimand interpretation explicitly so readers do not generalize to independent-generation contrasts without new experiments.

Statistical regime, power, and limitations of inference. Our preregistered power rationale targeted detection of moderate absolute paired increases (~ 10 – 15 percentage points) at $N = 200$ under mid-range baseline assumptions (~ 45 – 65%). The realized baseline success rate ($199/200 = 99.5\%$) places the analysis in a sparse-discordant regime where the exact McNemar p -value gives limited directional evidence ($p = 1.0$ for our observed discordants). The paired-bootstrap CI $[0.00, 0.015]$ provides a bounded estimate of plausible effect sizes compatible with the data and shows the dataset is consistent with at most small absolute improvements under our repair-only design. This underscores that unexpectedly high baseline performance can substantially reduce realized inferential sensitivity for small marginal effects. Practically, when baseline acceptance is already near one, corrective repairs can still be operationally useful for rare but high-consequence failures; however, the present data quantify only that such rare corrections occurred at least once in the preregistered sample and do not permit precise frequentist inference about very small effect magnitudes beyond the bootstrap bounds reported.

Mechanistic interpretation of the single repair. The archived validator_trace shows the repaired case involved a multi-step ordering violation across multiple interfaces and required_command_count = 9. The repair used by the guarded pipeline enacted a localized reordering consistent with the task manifest’s required_sequence and produced validation_after.valid = true. This pattern aligns with our exploratory hypothesis (H3) that multi-device sequencing and inter-device

dependency rules concentrate residual failures. From a mechanistic standpoint, verifier-encoded ordering constraints are a concretely checkable invariant; when the initial candidate contains an expressible sequence breach, a bounded repair that reorders commands according to manifest constraints can correct the fault. That corrective potential depends directly on (a) the verifier’s ability to detect an expressible invariant breach and (b) the repair routine’s capacity to deterministically construct a corrected sequence without introducing new invariant violations.

Operational implications and bounded claims. Artifact-grounded evidence supports a narrow operational conclusion: in this synthetic deterministic task bank using gpt-5-mini and deterministic_netops_validator_v5, a verifier-triggered bounded repair corrected a localized multi-step ordering fault in one extreme task. This is direct evidence that verifier-guided bounded repair can remedy certain ordering/dependency violations in generated configurations when those violations are expressible in the task manifest’s invariants. It is not evidence that bounded repair will correct omissions only observable at dataplane runtime, or that guarded repair will address adversarially poisoned contexts; those remain open evaluation axes requiring live emulation, multi-model studies, and adversarial contamination experiments. Moreover, because our repair allowance was limited to a single automatic repair call per task, the observed effect quantifies what a minimally permissive guarded pipeline can accomplish; pipelines that permit multiple repair iterations, richer external-tool synthesis, or human-in-the-loop adjudication may achieve larger gains but were not evaluated here.

Threats to validity and generalization (expanded). Internal validity: the paired shared_initial_candidate design isolates repair effects but does not address prompt sensitivity, model sampling variability, or the potential for different initial candidates under guarded prompting strategies. The model_configuration.json records temperature = null (unset), which is not evidence of deterministic sampling; nondeterminism in hosted-model generation remains a background condition that the shared_initial_candidate semantics mitigated for the paired comparison but did not eliminate as a systemic source of variance. Construct validity: the verifier enforces encoded syntactic and ordering invariants but cannot capture behaviors observable only through dataplane testing (e.g., runtime routing convergence or vendor-specific command side-effects). External validity: the experiment used a single model (gpt-5-mini) and a synthetic, deterministic task bank; results do not imply cross-model or production-device generality. Conclusion validity: with a single discordant pair the realized inferential leverage for small repair-only effects is weak; the bootstrap CI quantifies that uncertainty and highlights the value of larger samples or targeted sampling focusing on high-difficulty strata for future studies.

Practical considerations for guarded pipeline design. The findings articulate two practical design lessons grounded in archived artifacts: (1) deterministic verifiers that explicitly encode ordering and group-level invariants can serve as ef-

fective selectors for when automated repair is necessary; the verifier’s binary valid flag cleanly triggers bounded repair without human intervention in our run. (2) When invariants are expressible and repairs are constrained (for example, limited to reordering or restricted field modifications), a single focused repair may suffice to recover validity for complex tasks. Both lessons depend on careful manifest design that encodes the relevant invariants and on repair primitives that are conservative (minimize unintended changes) and auditable.

Reproducibility and next evaluation axes. The archived execution and analysis artifacts provide a complete trace for re-analysis and extension: task manifests with required_sequence encodings, per-episode validator_trace outputs, model-call accounting, and bootstrap parameters (seed = 1729, resamples = 10000). To better estimate small repair-only effects or to evaluate alternative guarded semantics, subsequent preregistered runs should consider (a) stratified oversampling of extreme/high-complexity tasks to increase discordant counts, (b) multi-model comparisons, and (c) relaxing shared_initial_candidate semantics to measure independent guarded first-pass benefits. Each such axis requires careful preregistration to avoid post-hoc selection and to preserve interpretability of paired or unpaired estimands.

VI. LIMITATIONS

- Synthetic deterministic task bank and verifier: results reflect correctness under the chosen synthetic intent templates and deterministic verifier and do not guarantee similar performance on live heterogeneous production devices or telemetry.
- Single-model experiment (gpt-5-mini): findings do not demonstrate multi-model generality.
- Shared_initial_candidate semantics: baseline and guarded conditions began from the same initial generation; the study measures verifier/repair effects only and does not evaluate independent first-pass generation contrasts.
- Limited adversarial/contamination testing: the run did not include systematic adversarial retrieval or poisoned-context attacks; such threats remain an open evaluation axis.
- Oracle coverage: the deterministic verifier enforces encoded invariants but may omit runtime behaviors and device-specific semantics observable only through live execution; external validity to live deployments is therefore limited.

VII. CONCLUSION

We executed a preregistered paired experiment measuring the marginal effect of a verifier-triggered repair under shared_initial_candidate semantics for LLM-generated network configurations. In 200 synthetic deterministic tasks with gpt-5-mini and deterministic_netops_validator_v5, baseline acceptance was 199/200 and guarded acceptance was 200/200; a single automated repair corrected a multi-device ordering violation. Paired absolute risk difference = 0.005 with paired-bootstrap 95% CI [0.00, 0.015] (10,000 resamples,

seed 1729) and exact two-sided McNemar $p = 1.0$. Artifact-grounded evidence therefore supports a constrained conclusion: verifier-guarded bounded repair can correct localized ordering faults in this synthetic verifier-backed setting. Broader operational claims require additional experiments: multi-model studies, independent-first-pass guarded semantics, adversarial contamination testing, and live-device or dataplane emulation to capture runtime semantics not modeled by the deterministic verifier.

DISCLOSURE STATEMENT

Disclosure: This run implemented preregistered study CAND-2026-001 and was executed autonomously after the autonomy lock. Declared model: gpt-5-mini (execution/model_configuration.json records temperature = null/unset; null/unset is not evidence of deterministic sampling or temperature = 0). Orchestration adapter: hosted_netops_gvr_v1. Exact initial master prompt archived at provenance/master_prompt.txt (SHA-256 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582b b39b15ba). Preregistration fixed generation_semantics = shared_initial_candidate (one sampled initial generation per task was deterministically reused by baseline and guarded episodes) and allowed at most one automated repair call per task. Model-call accounting in the archived execution manifest reflects 200 shared initial generation calls and 1 repair call (201 model calls total). The deterministic verifier used was deterministic_netops_validator_v5. Humans selected the broad topic family and constrained the initial master prompt prior to the autonomy lock as permitted by the track; no human manually edited technical manuscript content after the autonomy lock. All provider traces, verifier logs, task manifests, model-configuration records, and analysis artifacts are archived in the execution bundle and indexed by the execution manifest.

REFERENCES

- [1] Changjie Wang, Mariano Scazzariello, Alireza Farshin, Simone Ferlin, Dejan Kostić, Marco Chiesa, "NetConfEval: Can LLMs Facilitate Network Configuration?", 2024, 10.1145/3656296
- [2] Ryan Beckett, Aarti Gupta, Ratul Mahajan, David Walker, "A General Approach to Network Configuration Verification", 2017, 10.1145/3098822.3098834
- [3] Jack Gallifant, Majid Afshar, Saleem Ameen, Yindalon Aphinyanaphongs, Shan Chen, Giovanni Cacciamani, Dina Demner-Fushman, Dmitriy Dligach, Roxana Daneshjou, Chrystinne Oliveira Fernandes, Lasse Hyldig Hansen, Adam Landman, Lisa Soleymani Lehmann, Liam G. McCoy, Timothy A. Miller, Amy C. Moreno, Nikolaj Munch, David Restrepo, Guergana Savova, Renato Umeton, Judy Wawira Gichoya, Professor Gary S. Collins, Karel G.M. Moons, Leo Anthony Celi, Danielle S. Bitterman, "The TRIPOD-LLM reporting guideline for studies using large language models", 2025, 10.1038/s41591-024-03425-5
- [4] Yunhe Li, "Test-in-the-loop LLM Repair: Verifiable Automated Program Repair on QuixBugs with a "Failing Test → Patch → Regression Test" Loop", 2024, 10.69987/jacs.2024.40206
- [5] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Zican Dong, Yupeng Hou, Beichen Zhang, Yingqian Min, Junjie Zhang, Peiyu Liu, Xiaolei Wang, Yifan Du, Yushuo Chen, Yushuo Chen, Zhipeng Chen, Jinhao Jiang, Ruiyang Ren, Yifan Li, Xinyu Tang, Peiyu Liu, Yiwen Hu, Jian-Yun Nie, Ji-Rong Wen, "A Survey of Large Language Models", 2026, 10.1007/s11704-026-60308-3

- [6] Haoran Dong, "Tool-Augmented Large Language Model Agents for Analysis: A Focused Study", 2026, 10.2139/ssrn.6647800